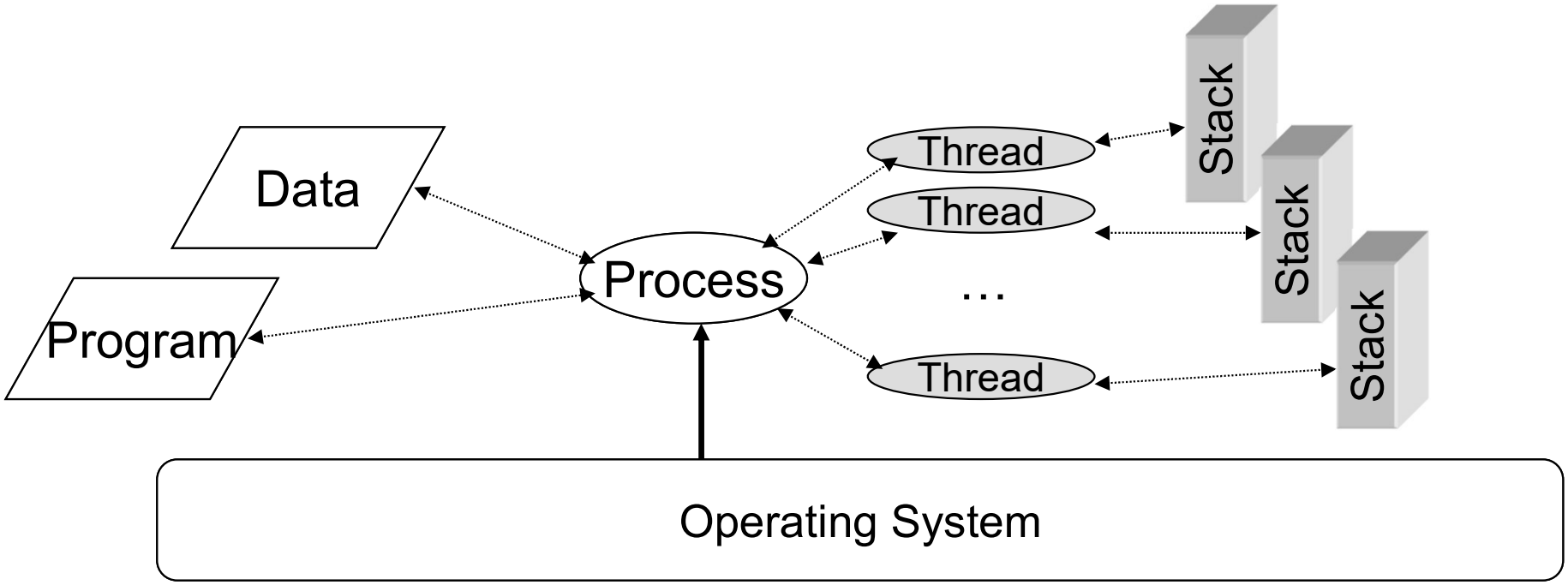
Threads

Processes and Threads

- A Process embodies two characteristics:
  - Resource ownership
    - Address space
    - I/O channels, devices, files
  - Scheduling/Execution
    - Follows execution path, interleaved with other processes
    - Execution State (Running, Ready, etc)
- What if we treat each independently?
  - Unit of resource ownership → process or task
  - Unit of Execution or Dispatch→ Thread or lightweight process

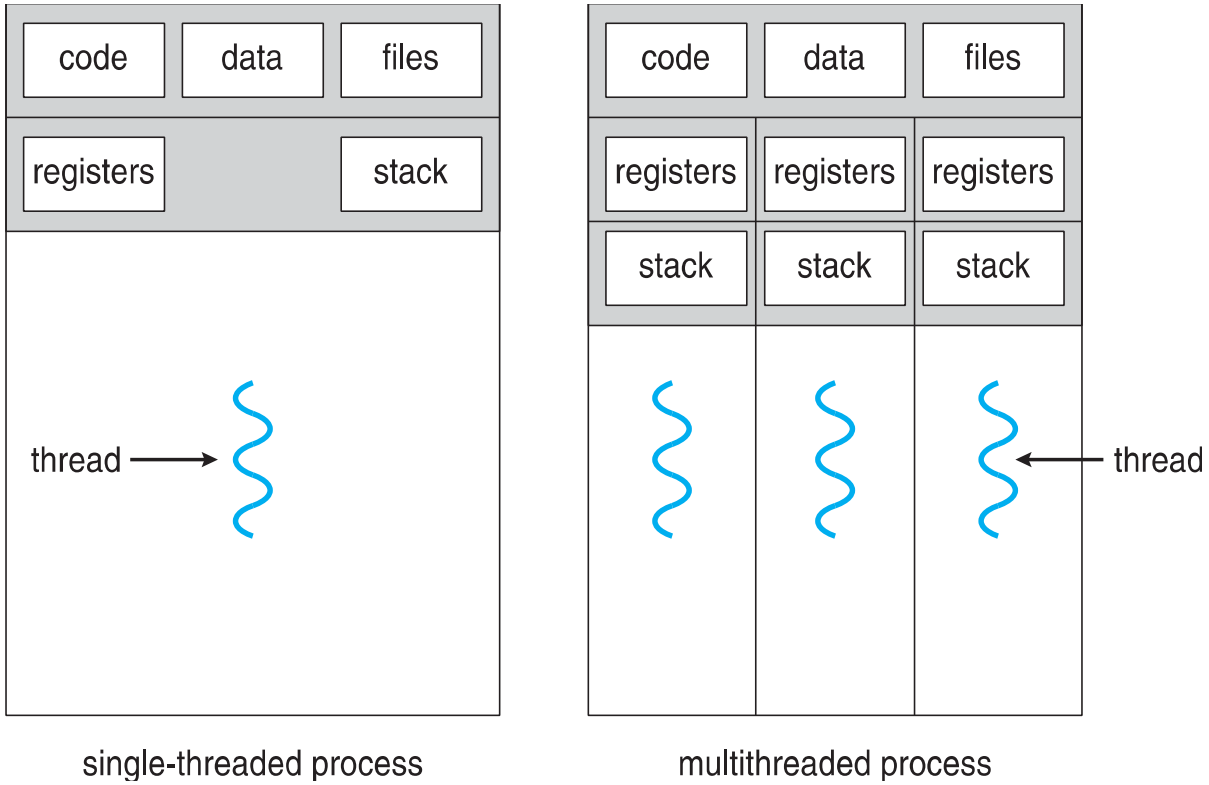
Processes and Threads

- ***Process*** is an infrastructure in which execution takes place → (address space + resources)
- ***Thread*** is a program in execution within a process context
  - Share resources of the process
  - Reside in the same address space
  - Have access to the same program and data
  - But, each thread has its own stack



Single and Multithreaded Processes

- A Thread is a basic unit of CPU utilization that forms the basis of multithreaded systems
  - it comprises a thread ID, a program counter, a register set, and a stack
  - It shares with other threads of the same process its code section, data section, and other OS resources, such as open files and signals
- A traditional (or ***heavyweight***) process has a single thread of control.



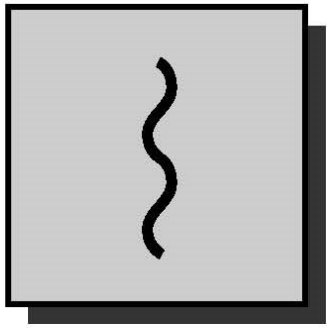
Multi-threading

- OS supports multiple, concurrent threads (paths of execution) within a single process
  - Traditional approach is single-threaded
- Examples:
  - MS-DOS supports a single user process and a single thread
  - UNIX supports multiple user processes but only supports one thread per process
  - Java run-time environment is a system of one process with multiple threads
  - Windows 2000 (W2K), Solaris, Linux, Mach, and OS/2 support multiple processes, each of which supports multiple threads

= instruction trace

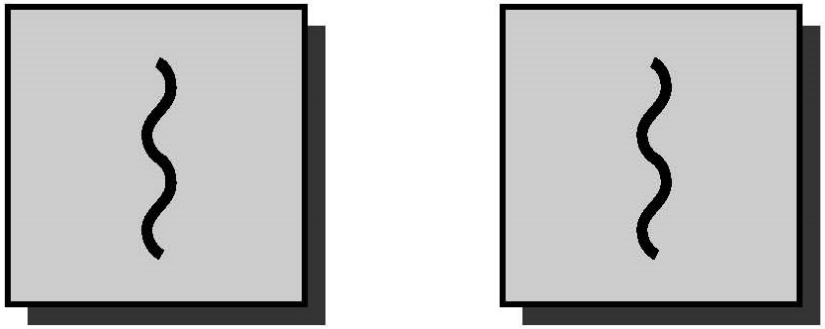
Single-threaded approaches

MS-DOS



one process
one thread

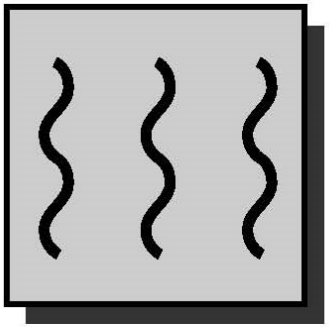
Some variants of UNIX



multiple processes
one thread per process

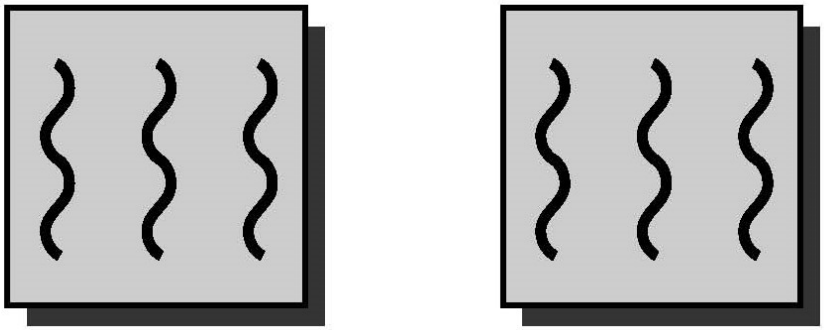
Multithreaded Approaches

Java Run Time Environment



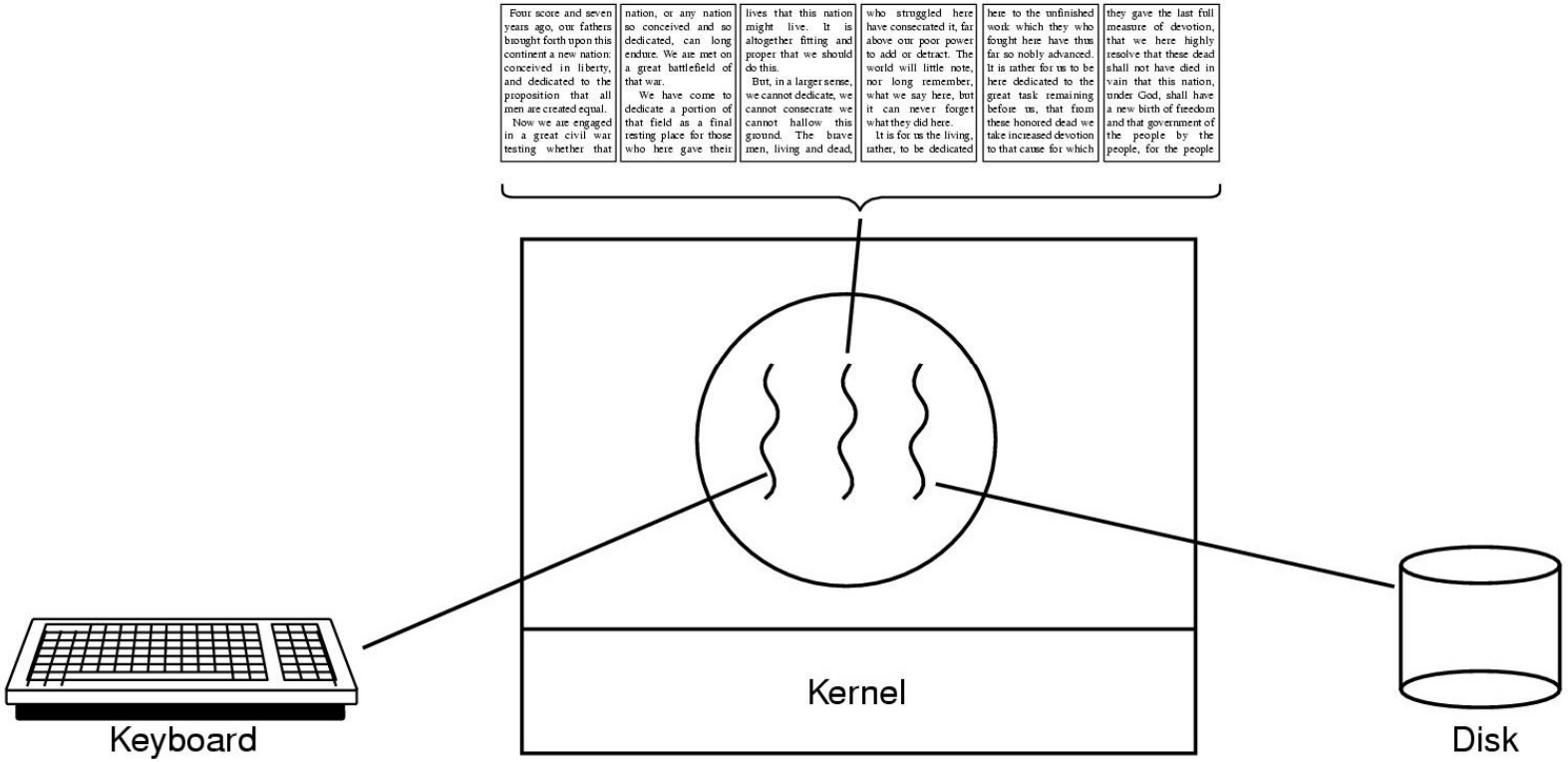
one process
multiple threads

Windows, Solaris, modern versions of UNIX



multiple processes
multiple threads per process

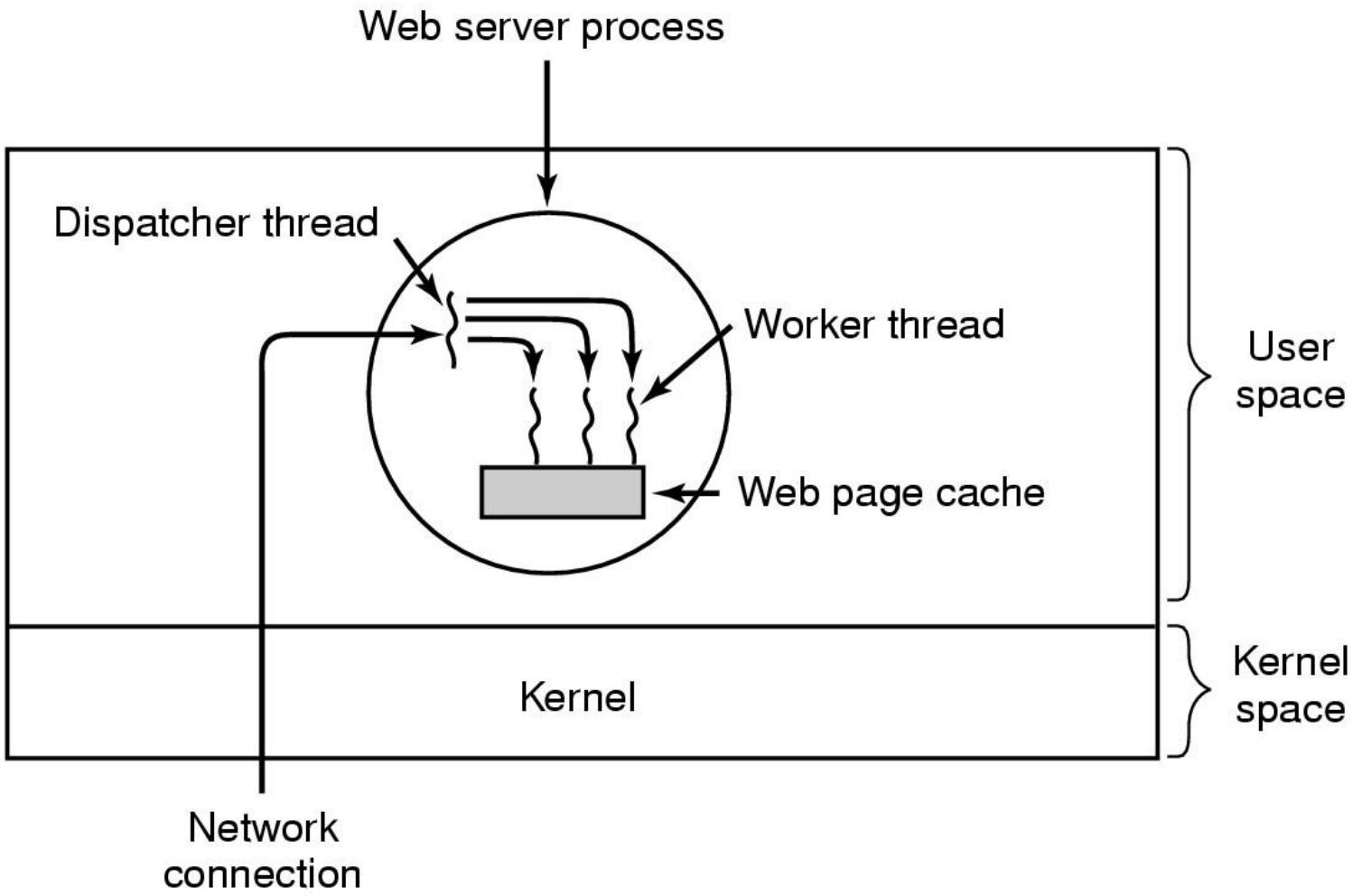
Thread Usage



A **word processor** with three threads

- The **first thread** just interacts with the user.
- The **second thread** reformats the document in the background.
- The **third thread** writes contents of RAM to disk periodically

Thread Usage



A multithreaded Web server

Threads

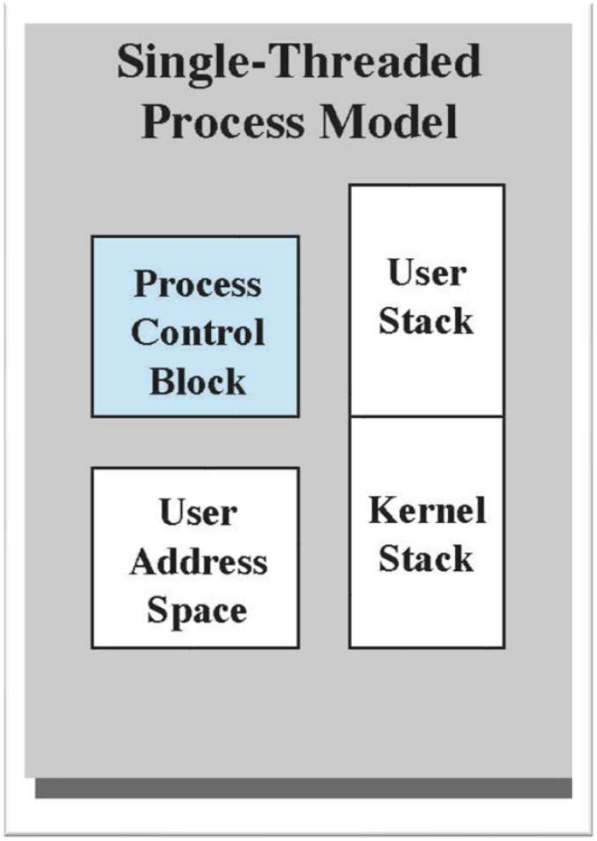
- Most modern applications are multithreaded
- If a process has multiple threads of control, it can perform more than one task at a time
- Multiple tasks with the application can be implemented by separate threads e.g.
  - Update display
  - Fetch data
  - Spell checking
  - Answer a network request
- Process creation is heavy-weight while thread creation is light-weight
- Can simplify code, increase efficiency
- Kernels are generally multithreaded

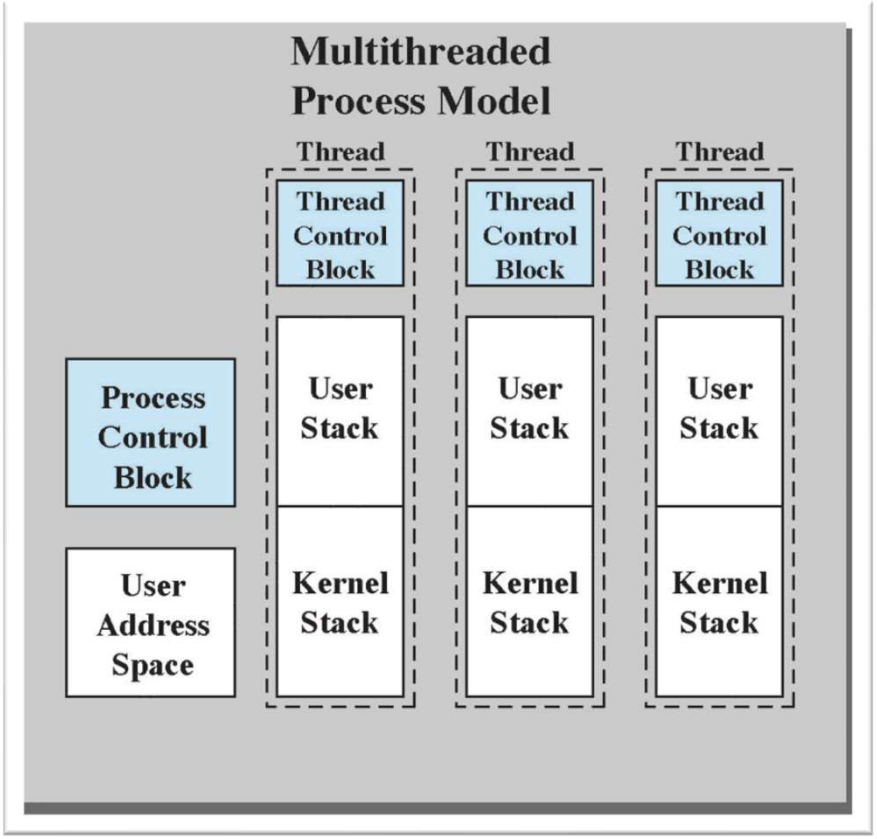
Threads

- Threads of a process share resources of the process
- Threads reside in the same address space and have access to the same data
  - When one thread alters an item of data in memory, other threads see the results if and when they access that item
  - If one thread opens a file with read privileges, other threads in the same process can also read from that file
- Scheduling and dispatching is done on thread basis
  - Execution state is maintained on thread level
- Some actions are global on process and all its threads
  - Suspension of a process – swapping
  - Termination of a process implies terminating all its threads

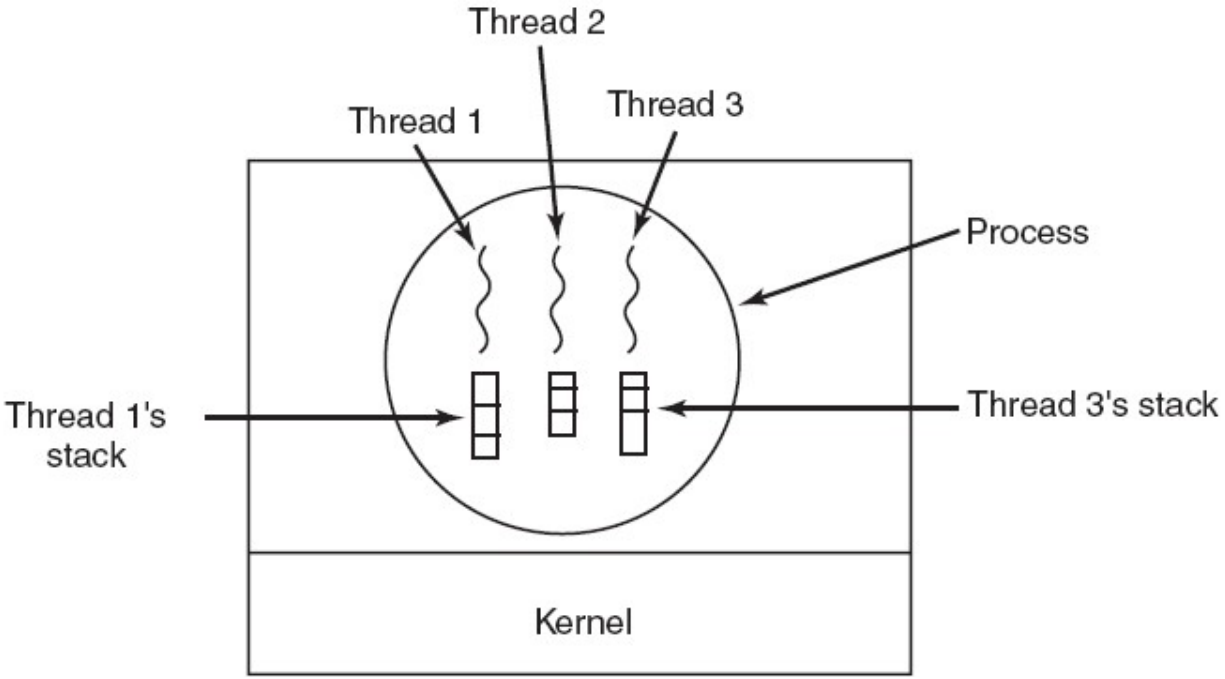
Threads vs Processes w.r.t. Process Management

- One way to view a thread is as an independent program counter operating **within** a process.
- Thread Control Block contains register values, priority, state, etc..





The Classical Thread Model



Per process items	**Per thread items**
Address space	Program counter
Global variables	Registers
Open files	Stack
Child processes	State
Pending alarms	
Signals and signal handlers	
Accounting information	

Benefits

- **Responsiveness** – may allow continued execution if part of process is blocked, especially important for user interfaces
- **Resource Sharing** – threads share resources of parent process, easier than shared memory or message passing
- **Efficiency**
  - Since threads within the same process share memory and files, they can **communicate** with each other **w/o** invoking the **kernel**
- **Economy**
  - Cheaper than process **creation (x30)**
  - Thread **switching** is lower overhead than context switching (x5)
  - Cheaper to **terminate** a thread than a process
- **Scalability** – process can take advantage of multiprocessor architectures

POSIX Threads

- POSIX (Portable Operating Systems based on UNIX)
- An IEEE API standard that includes a standard for a thread API.
- Libraries implementing the POSIX Threads standard are often named *Pthreads.*
- Supported in most platforms
- Some of the Pthreads function calls.

Thread call	Description
Pthread_create	Create a new thread
Pthread_exit	Terminate the calling thread
Pthread_join	Wait for a specific thread to exit
Pthread_yield	Release the CPU to let another thread run
Pthread_attr_init	Create and initialize a thread's attribute structure
Pthread_attr_destroy	Remove a thread's attribute structure

POSIX Threads - Example

. . .

```
int main(int argc, char *argv[])
{
        /* The main program creates 10 threads and then exits. */
        pthread_t threads[NUMBER_OF_THREADS];
        int status, i;

        for(i=0; i < NUMBER_OF_THREADS; i++) {
                printf("Main here. Creating thread %d0, i);
                status = pthread_create(&threads[i], NULL, print_hello_world, (void *)i);

                if (status != 0) {
                        printf("Oops. pthread_create returned error code %d0, status);
                        exit(-1);
                }
        }
        exit(NULL);
}
```

POSIX Threads - Example

```
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>

#define NUMBER_OF_THREADS     10

void *print_hello_world(void *tid)
{
        /* This function prints the thread's identifier and then exits. */
        printf("Hello World. Greetings from thread %d0, tid);
        pthread_exit(NULL);
}          . . .
```

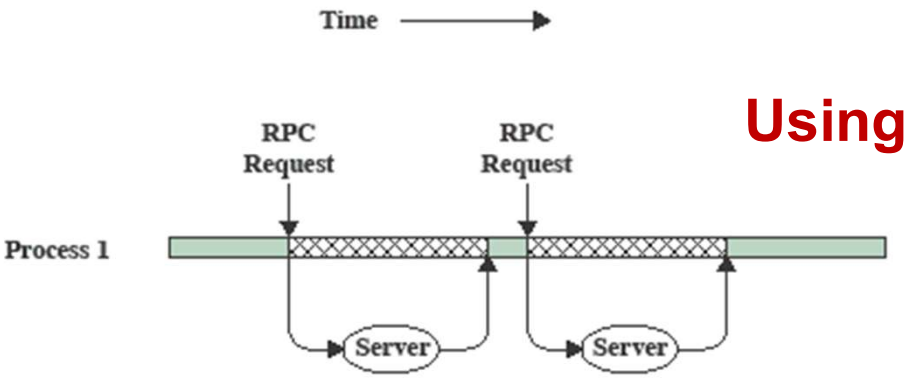
Thread Execution States

- Several actions affect all threads in a process e.g.
  - Suspending a process involves suspending all its threads
  - Termination of a process, terminates all threads within the process
- Key Thread states: Running, Ready and Blocked
- Thread Operations
  - Spawn – create another thread (with register context and stack)
  - Block - Waiting for an event (saving user registers, PC and stack)
    - Would blocking a thread block all other threads of a process?
      - Generally a thread can block without blocking the remaining threads in the process
      - Allow the process to start two operations at once, each thread blocks on the appropriate event
  - Unblock - Event happened. Thread moves to Ready queue
  - Finish - thread completed (Deallocate register context and stacks)

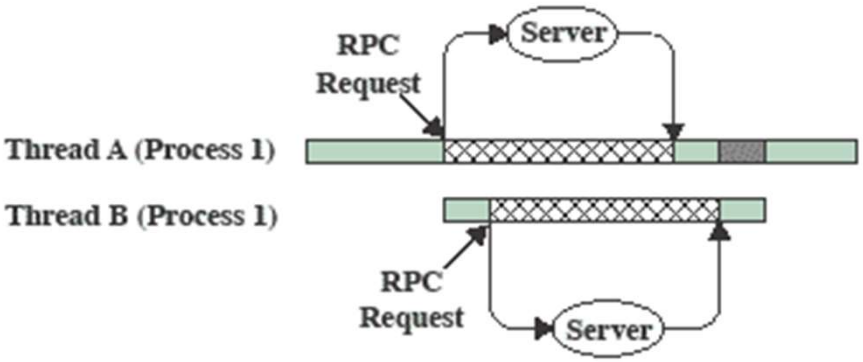
Threads and RPC

Consider a program that performs two remote procedure calls (RPCs) to two different hosts to obtain a combined result.

RPC Using Single Thread

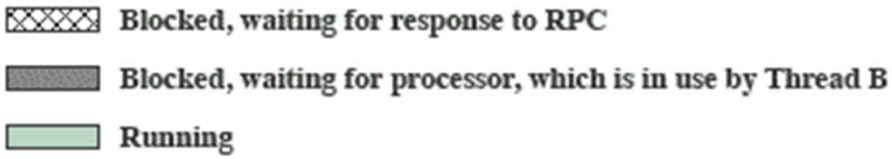


(a) RPC Using Single Thread

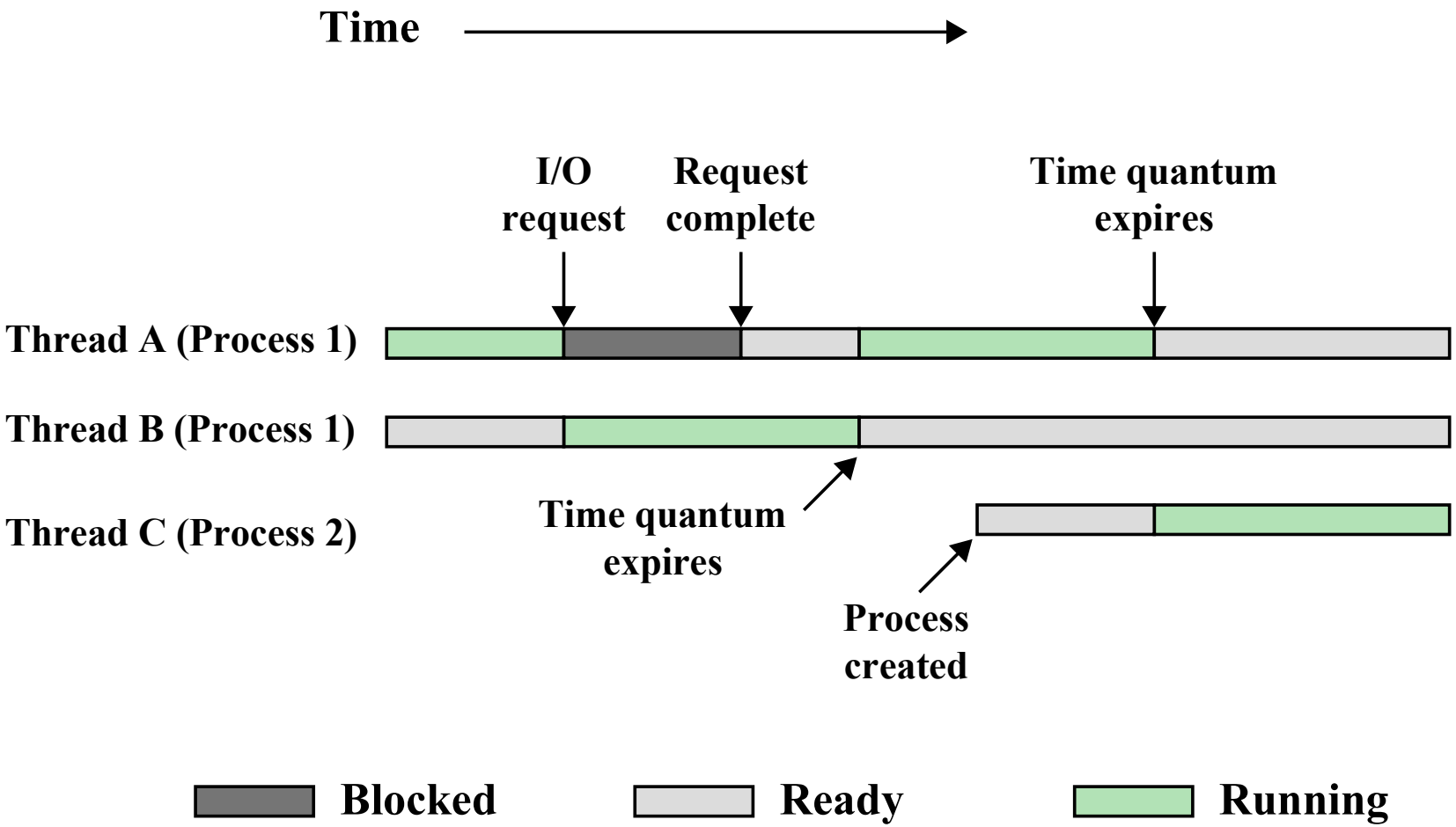


(b) RPC Using One Thread per Server (on a uniprocessor)

RPC Using One Thread per Server



Multithreading on a Uniprocessor



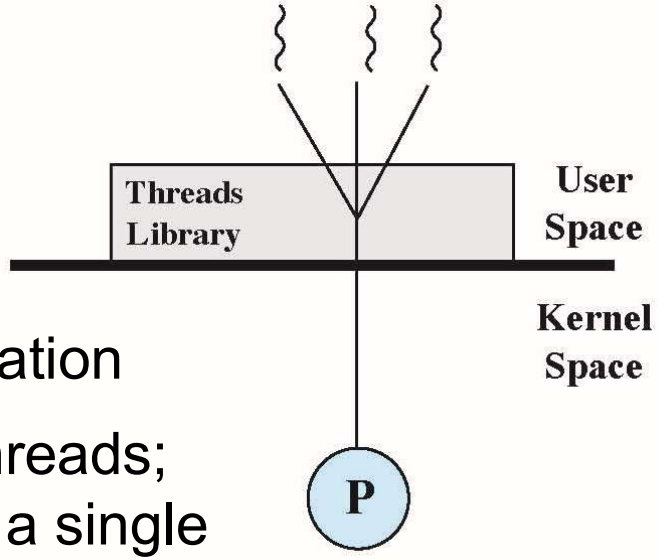
On a uniprocessor, multiprogramming enables the interleaving of multiple threads within multiple processes.

Execution passes from one thread to another either when the running thread is blocked or its time slice is exhausted.

How are Threads Implemented

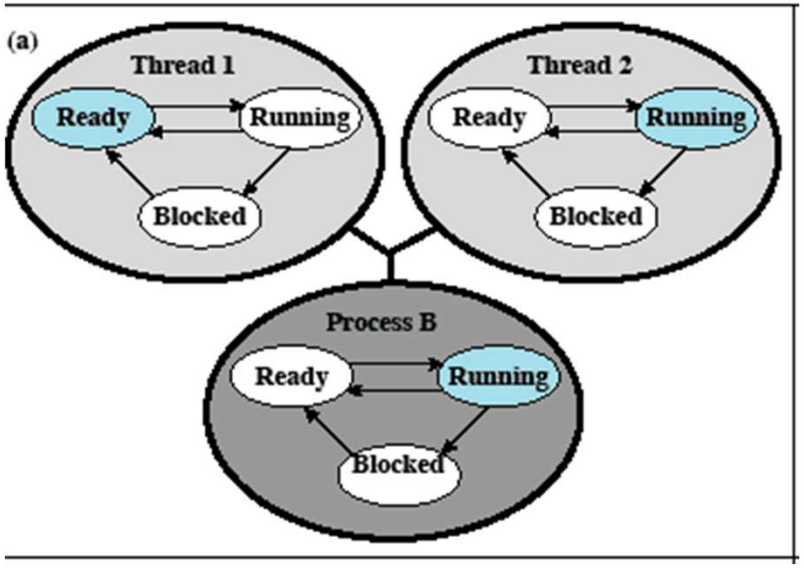
- **User Level Threads (ULT)**
  - Management done by user-level threads library
  - Three primary thread libraries:
    - POSIX **Pthreads**
    - Windows threads
    - Java threads
- **Kernel Level Threads (KLT)** (called Lightweight Processes)
  - Supported by the Kernel
  - Examples – virtually all general purpose operating systems:
    - Windows
    - Solaris
    - Linux
    - Tru64 UNIX
    - Mac OS X

User-Level Threads (ULT)



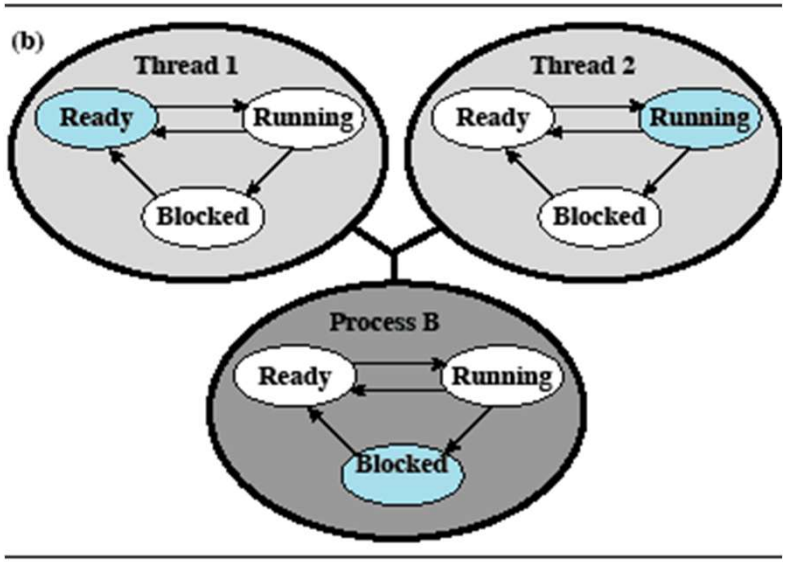
- All thread management is done by the application
- The kernel is not aware of the existence of threads; schedules the process as a unit and assigns a single execution state (Ready, Running, Blocked, etc.)
- Any application can be programmed to be multithreaded by using a ***threads library*** which contains code for **creating** and **destroying** threads, for passing messages and data between threads, for **scheduling** thread execution, and for saving and restoring thread contexts.
- An application starts with a single thread, spawns new threads and performs the scheduling of those threads. Threads will execute on processor ***only when*** the process is in Running state

Relationships between ULT and Process States



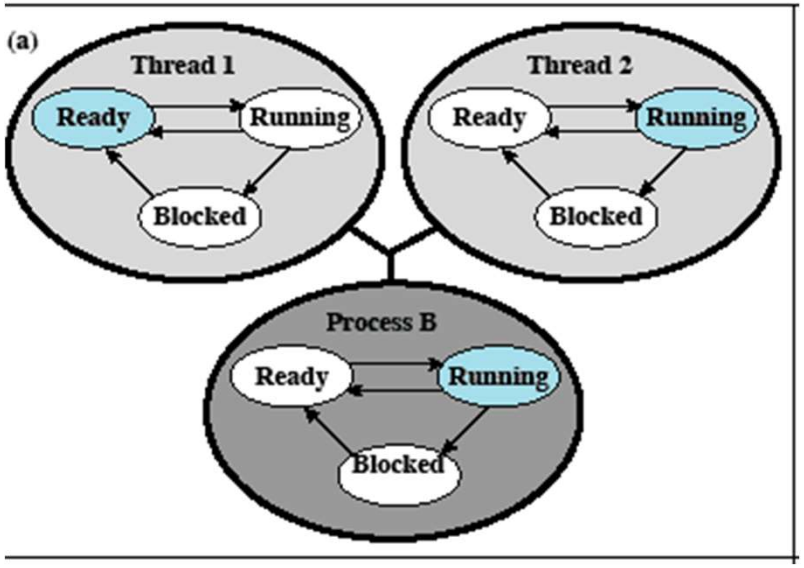
Suppose that process B, spawned two threads , and is executing in its Thread 2

Possible occurrences:



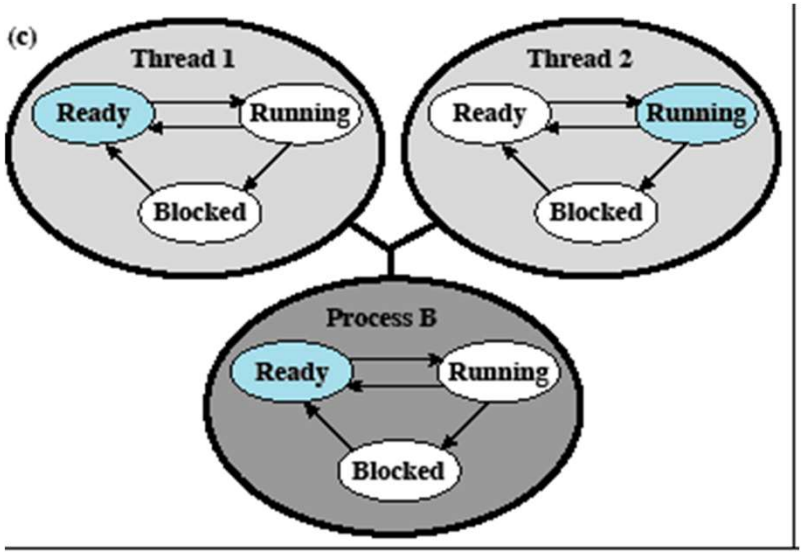
- Thread 2 makes a blocking system call
- Control transferred to Kernel, Blocks B and switch to another process
- Thread2 is perceived by the Thread Library as Running but it is not

Relationships between ULT and Process States



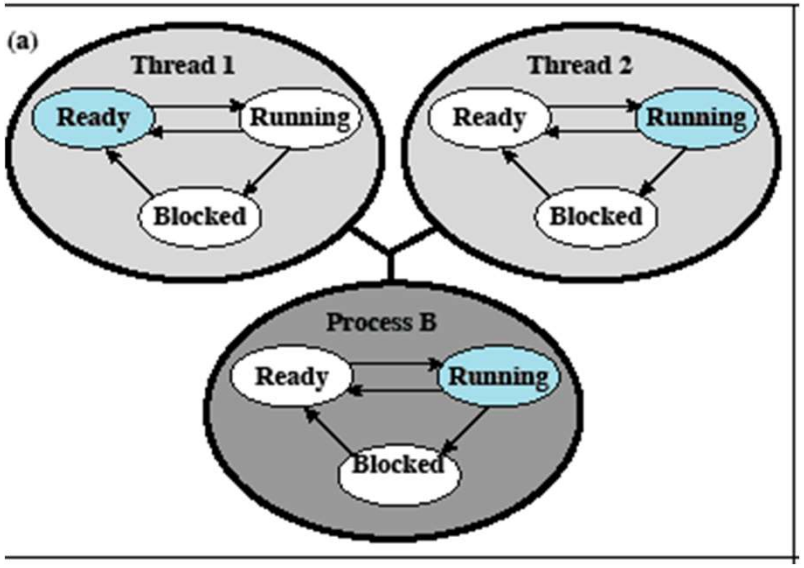
Suppose that process B, spawned two threads , and is executing in its Thread 2

Possible occurrences:



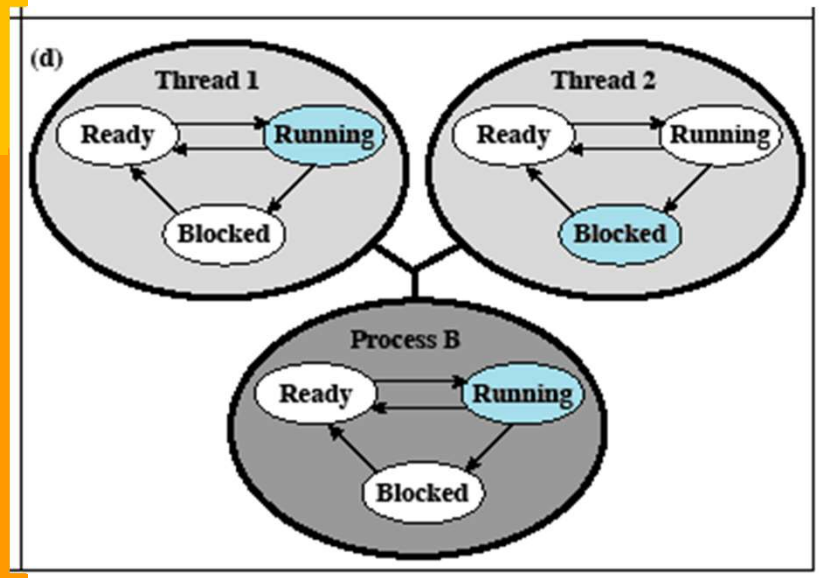
- Process B exhausts its time slice
- B is moved to Ready state and Kernel switches to another process
- Thread2 is perceived by the Thread Library as Running but it is not

Relationships between ULT and Process States



Suppose that process B, spawned two threads , and is executing in its Thread 2

Possible occurrences:

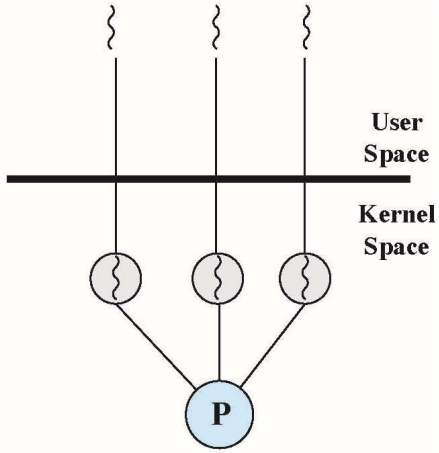


- Thread2 needs some action performed by Thread1
- Thread 2 enters a Blocked state and Thread1 moves from Ready to Running
- B remains in Running state

User-Level Threads (ULT)

- **Advantages**
  - Thread switching is done in user space and does not need kernel mode. Saving overhead of two mode switches (user → kernel; kernel → user)
  - Scheduling can be application specific, tailored to its requirement regardless of the scheduling algorithm of OS
  - ULT can run on any OS, no changes needed to the Kernel
- **Disadvantages:**
  - When a ULT executes a blocking system call, all of the threads within the same process are blocked
  - Can not take advantage of multiprocessing

Kernel-Level Threads (KLT)



- All of the work of *thread management* is done by the kernel. i.e. no thread management code in the program.
- Kernel maintains context information for the process as a whole and for individual threads within the process
- Scheduling is done by the kernel on a thread basis

Advantages:

- Kernel can simultaneously schedule multiple threads from the same process on multiple processors
- If one thread in a process is blocked, the kernel can schedule another thread of the same process
- Kernel routines themselves can be multithreaded

Disadvantage: transfer of control from one thread to the next in the same process requires a mode switch to the kernel

Performance Comparison between Process, ULT and KLT

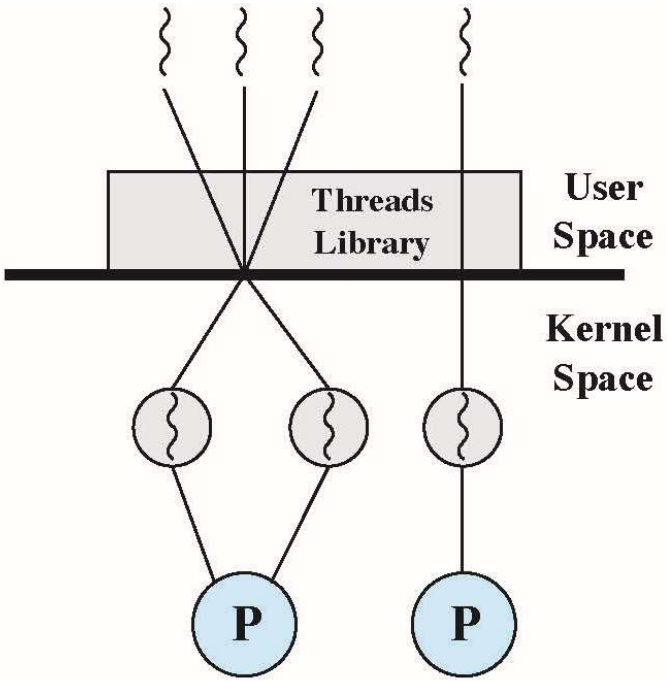
- Measurements taken on VAX Running UNIX-Like OS (latencies)
- Null Fork
  - The time to create, schedule, execute, and complete a process/thread that invokes the null procedure
  - represents the overhead of forking a process/thread
- Signal-Wait
  - the time for a process/thread to signal a waiting process/thread and then wait on a condition
  - i.e. the overhead of synchronizing two processes/threads
- order of magnitude between ULT and KLT and between KLT and processes

Operation	User-Level Threads	Kernel-Level Threads	Processes
Null Fork	34	948	11,300
Signal Wait	37	441	1,840

Thread and Process Operation Latencies (μs)

Combined Approaches

- Example: **SUN Solaris**
- Thread *creation*, bulk of *scheduling* and *synchronization* are done in the user space
- ULTs from a single application are mapped onto equal (or smaller) number of KLTs
- The programmer adjust # of KLTs for an application and processor to achieve the best overall results.
- Multiple threads within the same application can run in parallel on multiple processors
- If one is blocked, another thread within the same application need not block
- Combine adv of ULT and KLT approaches



Relationship between Threads & Processes

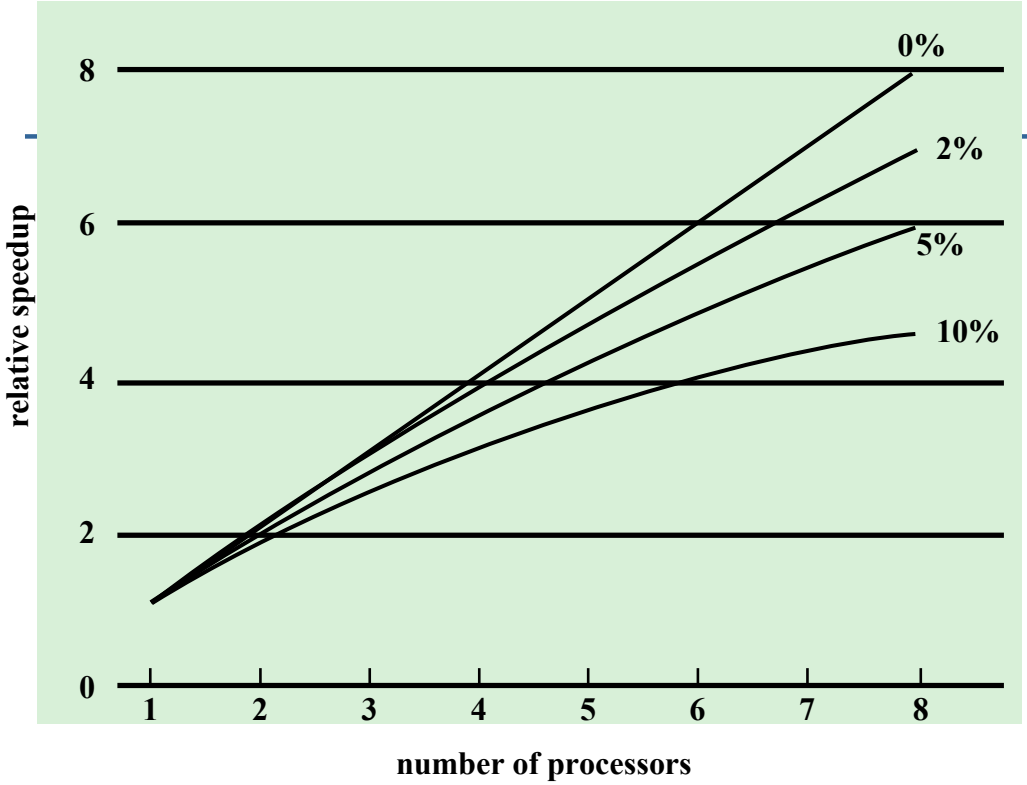
Threads:Processes	Description	Example Systems
1:1	Each thread of execution is a unique process with its own address space and resources.	Traditional UNIX implementations
M:1	A process defines an address space and dynamic resource ownership. Multiple threads may be created and executed within that process.	Windows NT, Solaris, Linux, OS/2, OS/390, MACH
1:M	A thread may migrate from one process environment to another. This allows a thread to be easily moved among distinct systems.	Ra (Clouds), Emerald
M:N	Combines attributes of M:1 and 1:M cases.	TRIX

Amdahl's Law

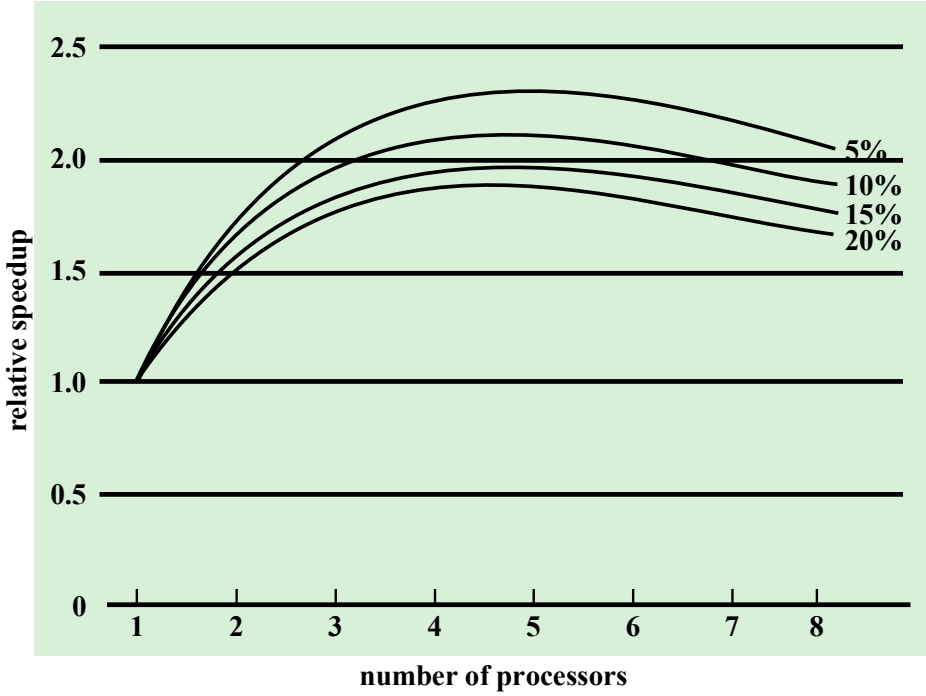
- Identifies performance gains from adding additional cores to an application that has both serial and parallel components
- ***S*** is serial portion of the application
- ***N*** processing cores

$$speedup \leq \frac{1}{S + \frac{(1-S)}{N}}$$

- That is, if application is 75% parallel / 25% serial, moving from 1 to 2 cores results in speedup of 1.6 times, moving to 4 cores speedup of 2.28 times
- As *N* approaches infinity, speedup approaches 1/*S. e.g.* if 40% if app is serial (S=0.4), then maximum speed up is 2.5 times
  Serial portion of an application has disproportionate effect on performance gained by adding additional cores
- But does the law take into account contemporary multicore systems?



(a) Speedup with 0%, 2%, 5%, and 10% sequential portions



(b) Speedup with overheads

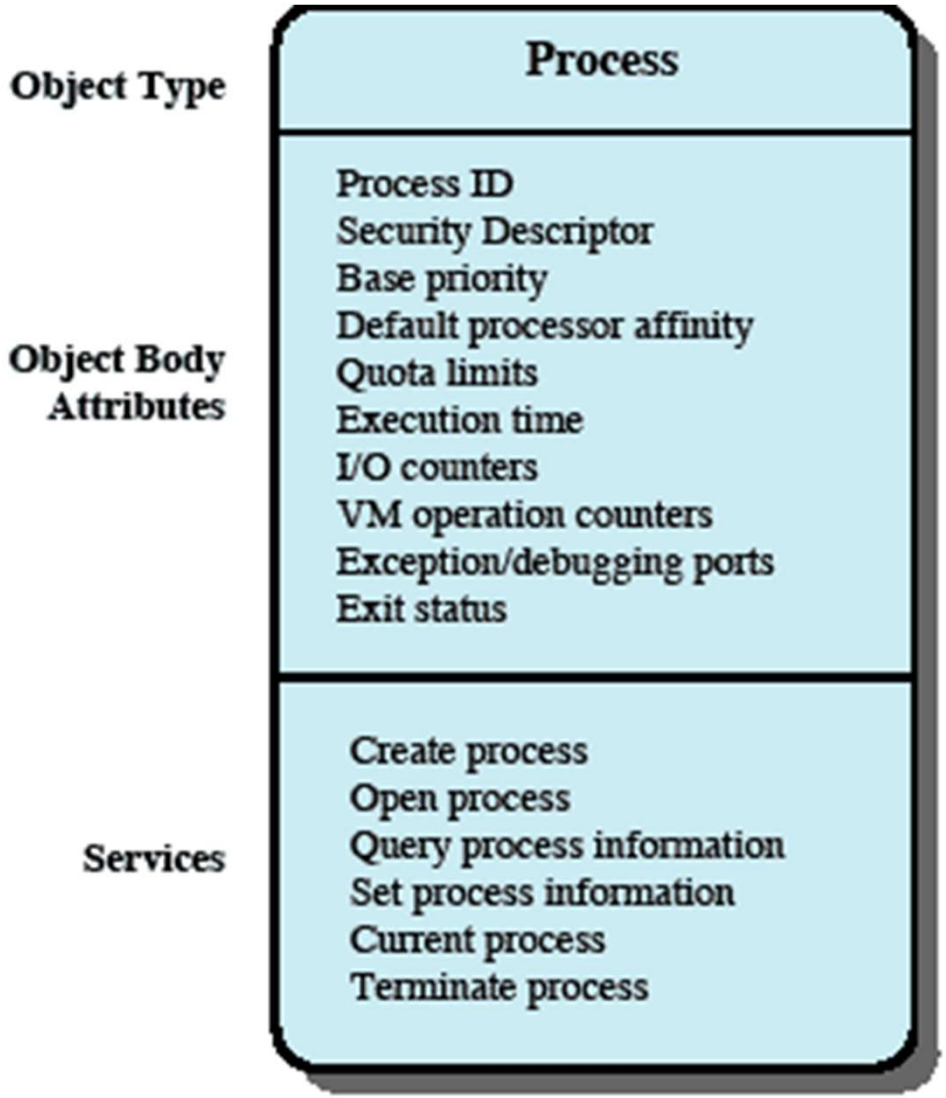
Figure 4.7 Performance Effect of Multiple Cores

Case Studies

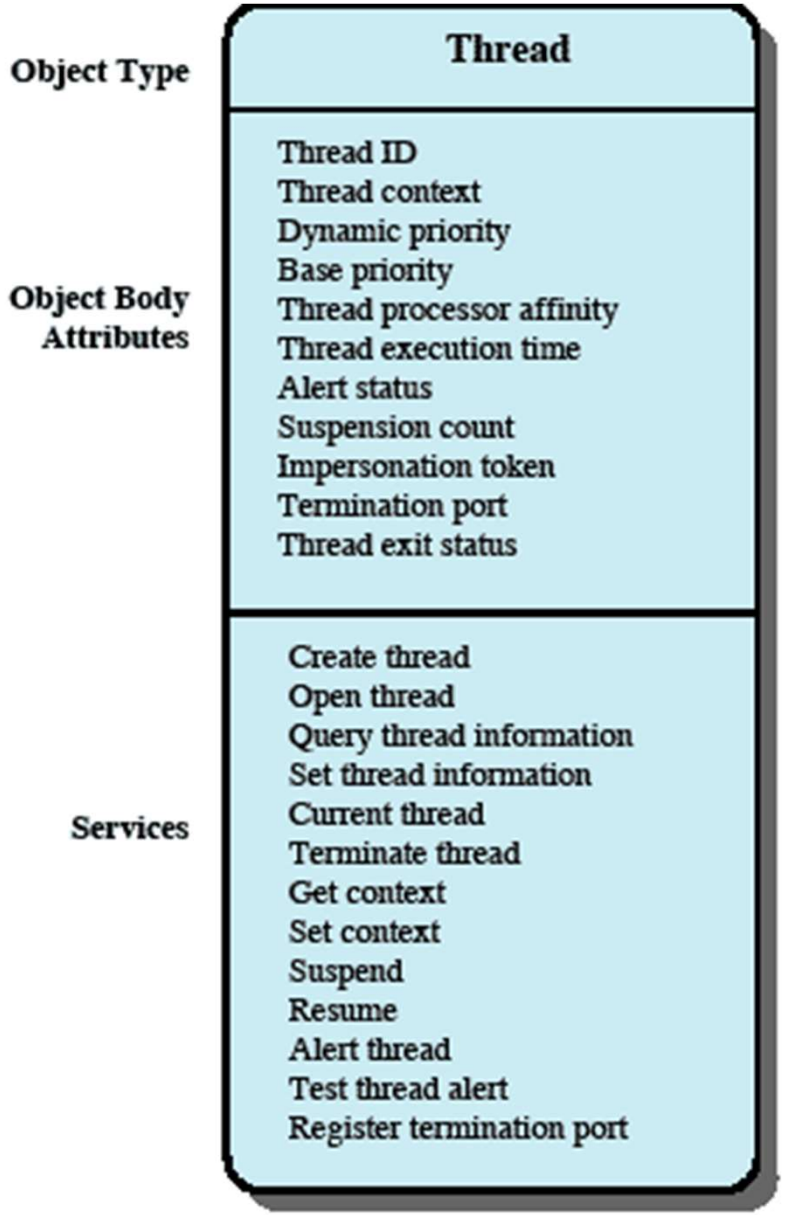
Windows

- Processes structure and services are simple and general purpose
  - Implemented as objects
  - An executable process may contain one or more threads
  - Processes/thread objects have built-in synchronization capabilities
- A process is an entity corresponding to a user job or application that owns resources, such as memory, and opens files.
- A thread is a dispatchable unit of work; executes sequentially; is interruptible, so that the processor can turn to another thread.
- A process must contain at least one thread to execute, that can create other threads
- Multithreading
  - Threads in different processes can execute concurrently
  - Multiple threads within same process may be allocated to separate processors
  - Threads within same process exchange information through shared address spaces and share resources

Windows Process and Thread Objects



(a) Process object



(b) Thread object

Table 4.3 Windows Process Object Attributes

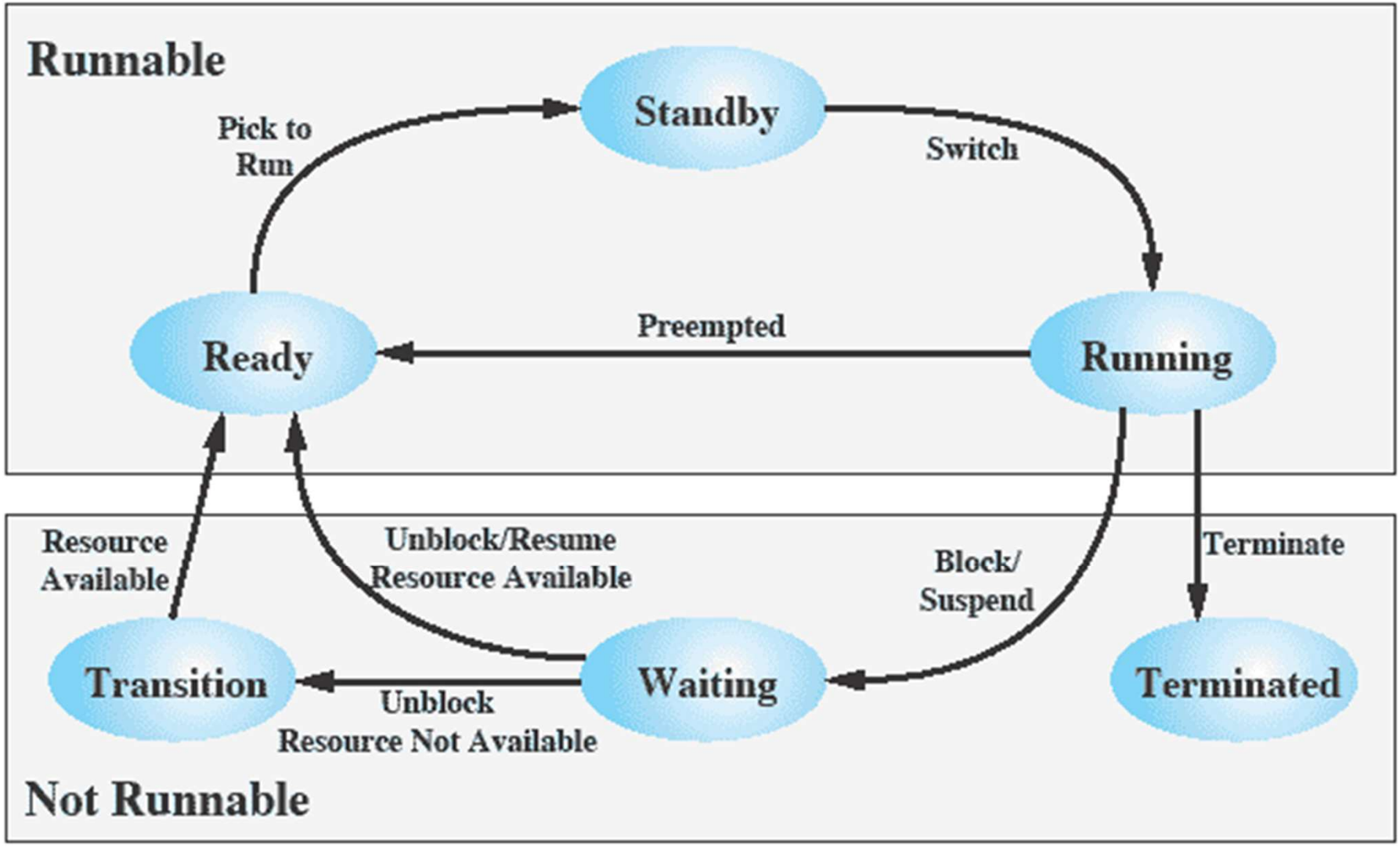
Process ID	A unique value that identifies the process to the operating system.
Security Descriptor	Describes who created an object, who can gain access to or use the object, and who is denied access to the object.
Base priority	A baseline execution priority for the process's threads.
Default processor affinity	The default set of processors on which the process's threads can run.
Quota limits	The maximum amount of paged and nonpaged system memory, paging file space, and processor time a user's processes can use.
Execution time	The total amount of time all threads in the process have executed.
I/O counters	Variables that record the number and type of I/O operations that the process's threads have performed.
VM operation counters	Variables that record the number and types of virtual memory operations that the process's threads have performed.
Exception/debugging ports	Interprocess communication channels to which the process manager sends a message when one of the process's threads causes an exception. Normally these are connected to environment subsystem and debugger processes, respectively.
Exit status	The reason for a process's termination.

some of the attributes of a thread resemble those of a process: the thread attribute value is derived from the process attribute value

Table 4.4 Windows Thread Object Attributes

Thread ID	A unique value that identifies a thread when it calls a server.
Thread context	The set of register values and other volatile data that defines the execution state of a thread.
Dynamic priority	The thread's execution priority at any given moment.
Base priority	The lower limit of the thread's dynamic priority.
Thread processor affinity	The set of processors on which the thread can run, which is a subset or all of the processor affinity of the thread's process.
Thread execution time	The cumulative amount of time a thread has executed in user mode and in kernel mode.
Alert status	A flag that indicates whether a waiting thread may execute an asynchronous procedure call.
Suspension count	The number of times the thread's execution has been suspended without being resumed.
Impersonation token	A temporary access token allowing a thread to perform operations on behalf of another process (used by subsystems).
Termination port	An interprocess communication channel to which the process manager sends a message when the thread terminates (used by subsystems).
Thread exit status	The reason for a thread's termination.

Windows Thread States



Windows SMP Support

- Threads can run on any processor
  - But an application can restrict affinity
- Kernel assigns a Ready thread to the next available processor
- Soft Affinity
  - The dispatcher tries to assign a Ready thread to the same processor it last ran on.
  - This helps reuse data still in that processor's memory caches from the previous execution of the thread.
- Hard Affinity
  - An application can restrict threads to certain processor

Solaris

- Support four separate thread concepts:
  - **Process**: includes user's address space, stack, and process control block.
  - **User-level threads**:
    - Implemented thru Thread Library
    - Invisible to the OS
    - a user-created unit of execution within a process
  - **Kernel threads:** the fundamental entities that can be scheduled to run on processors
  - **Lightweight processes:**
    - Can be viewed as a mapping between ULTs and kernel threads.
    - Each LWP supports ULT and maps to one kernel thread.
    - LWPs are scheduled by the kernel independently

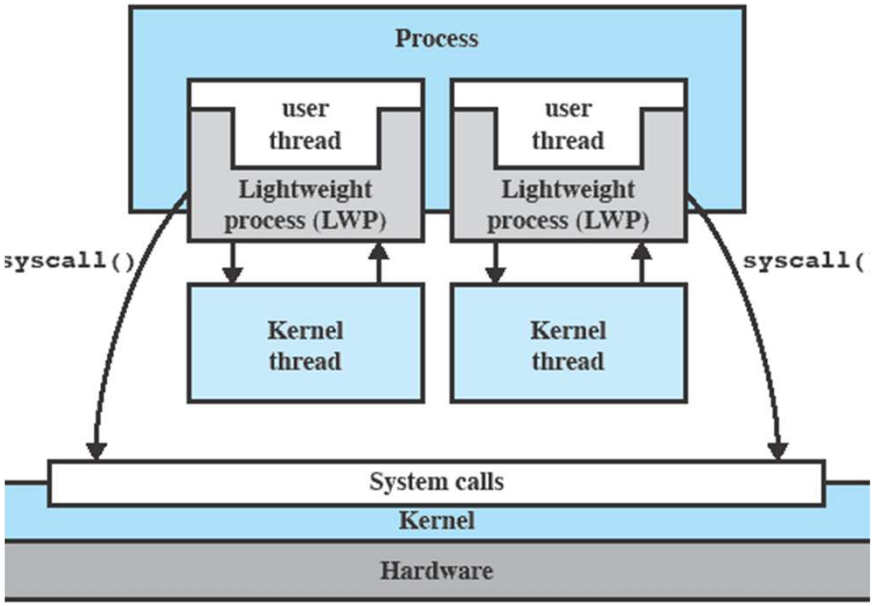


Figure 4.15 Processes and Threads in Solaris [MCDO07]

Solaris

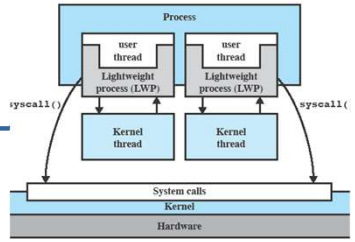
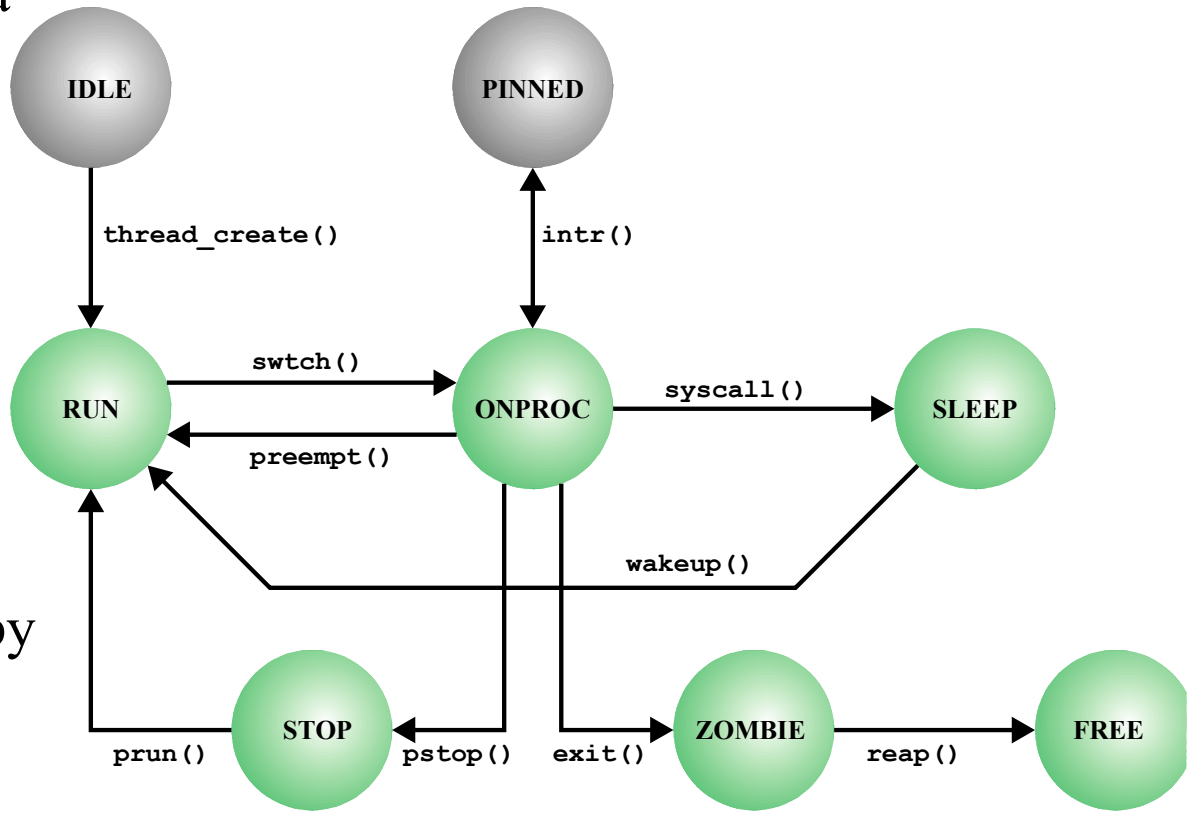


Figure 4.15 Processes and Threads in Solaris [MCDO07]

- There is always exactly one Kernel thread for each LWP
- A process may consist of a single ULT bound to a single LWP → single thread of execution → no concurrency
- If an application requires concurrency
  - its process contains multiple threads ULTs
  - each bound to a single LWP
  - which in turn are each bound to a single kernel thread
- Adv of 3-level thread structure (ULT, LWP, Kernel threads)
  - Facilitates thread management by OS since concurrency and execution is managed by kernel threads
  - Provide clean interface to applications: ULT interface can be a standard thread library
  - Application has access to hardware thru API

Solaris Thread States

•RUN: The thread is ready to execute
•ONPROC: thread is executing on a processor
•SLEEP: The thread is blocked.
• STOP: The thread is stopped.
• ZOMBIE: The thread has terminated.
• FREE: Thread resources are released



• ONPROC → RUN if preempted by a higher-priority thread or time-slicing
• ONPROC → SLEEP if it is waiting for an event
•A thread enters STOP state if its process is stopped; e.g. for debugging

Linux Processes

- A process in Linux is represented by a task_struct data structure
- This contains a number of categories including:
  - State The execution state of the process
  - Scheduling information: A process can be normal or real time and has a priority
  - Identifiers
  - Links: a process includes a link to its parent process, its siblings and children
  - Times and timers: Includes process creation time, amount of processor consumed by the process and interval timers.
  - File system: pointers to any files opened by this process
  - Address space: Defines the virtual address space assigned to this process
  - Processor-specific context: The registers and stack information that constitute the context of this process.

Linux Process/Thread Model

Running: one of two states Executing or Ready
Interruptible: blocked; waiting for an event, the availability of a resource, or a signal from another process
Uninterruptible: Another blocked state: a process is waiting directly on hardware conditions and therefore will not handle any signals
Stopped: The process has been halted; resume thru positive action from another process e.g debugging
Zombie: process has been terminated, but still has its PCB in process table

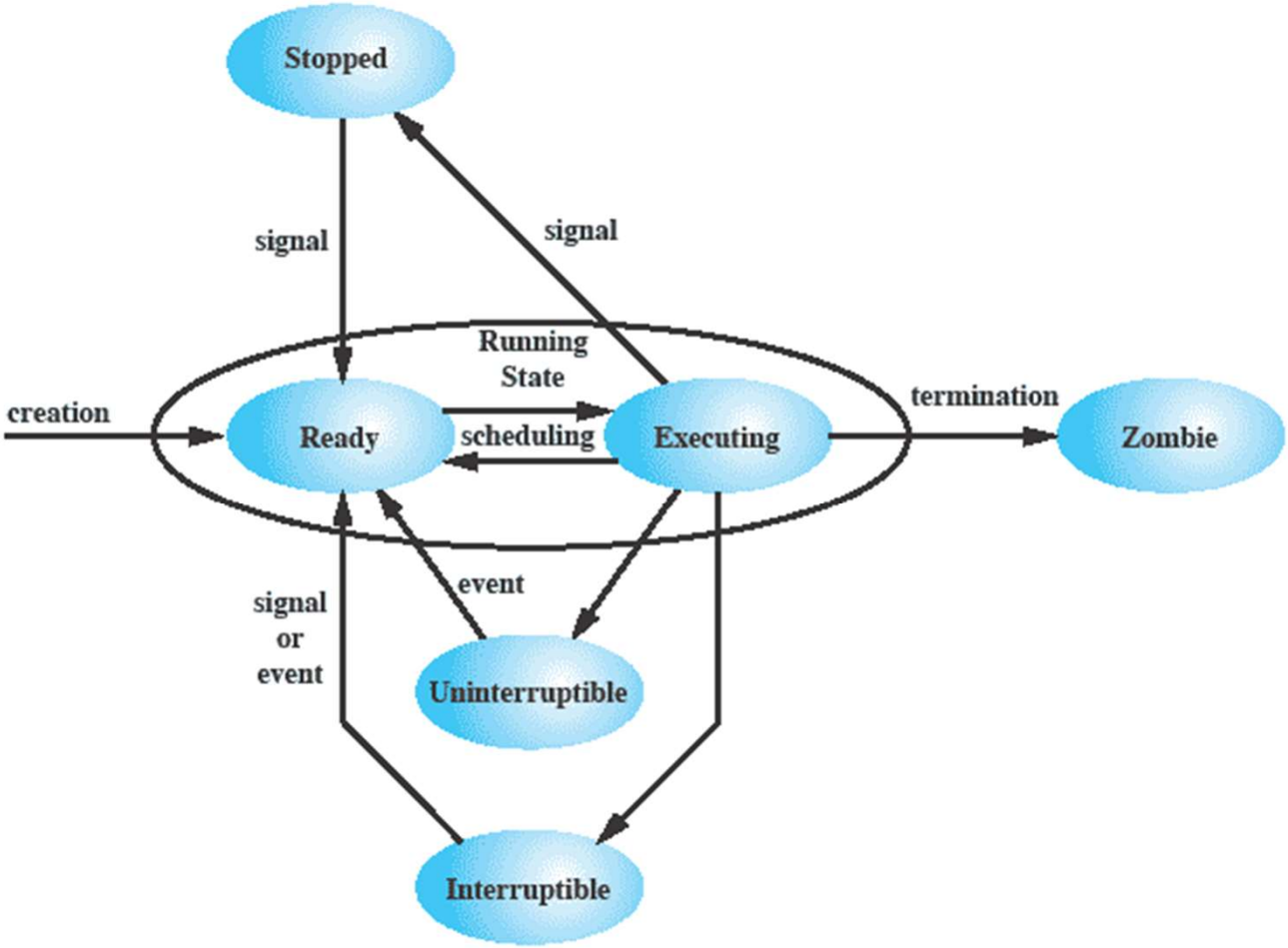


Figure 4.18 Linux Process/Thread Model

Linux Threads

- Older Linux:
  - No support for multithreading
  - Applications written with user level library functions POSIX
    - All threads mapping to a single kernel-level process
- Modern Linux
  - Mechanism similar to LWP of Solaris
  - ULT threads are mapped to Kernel level processes
  - Enable processes to share resources
  - A new process created thru clone() - no separate type for threads
    - Copy attribute of current process
    - Share resources e.g. files, signals and virtual memory
    - Kernel to perform a switch: checks if address space is the same, just jump to another location of code. No context switching
    - Do not share user stacks

POSIX Thread Library (Linux)

http://web.mst.edu/~ercal/284/ThreadExamples/thread_handout.doc

- Thread creation

  pthread_create()
- Synchronization

  pthread_mutex_lock(), pthread_mutex_unlock(), pthread_mutex_trylock()
- Suspension

  pthread_cond_wait(), pthread_cond_signal(), pthread_join()
- Yielding

  pthread_yield()
- Termination

  pthread_exit()

Android Process and Thread Management

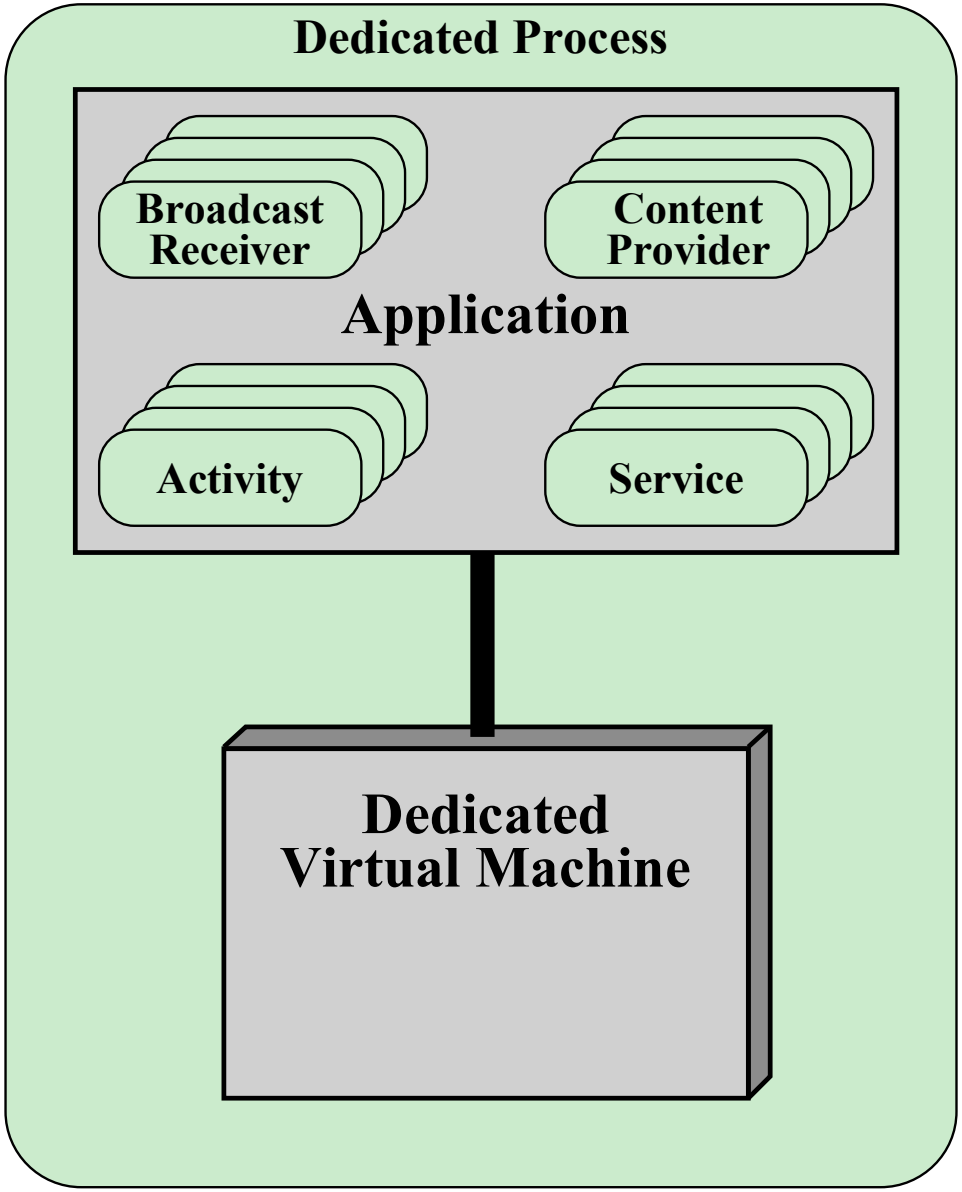
- Each Android application consists of one or more instance of one or more of four types of application components
- Each component performs a distinct role in the overall application behavior, and each component can be activated independently within the application and even by other applications
- Four types of components:
  - Activities
  - Services
  - Content providers
  - Broadcast receivers

Android Process and Thread Management

- Activities
  - an application component that provides a screen with which users can interact e.g. show list of emails, compose email, read email, etc..
- Services
  - used to perform background operations
  - Ex: a service might create a thread to play music, fetch data over the network without blocking user interaction
  - There are system services that run for the entire lifetime of the Android system, e.g Power Manager, Battery, and Vibrator services
- Content providers
  - acts as an interface to application data that can be used by the application e.g. NotePad uses a content provider to save notes
- Broadcast receivers
  - responds to system-wide broadcast announcements e.g. low battery warning

Android Application

- Each application runs on its own dedicated virtual machine and its own single process
- This approach, referred to as the **sandboxing model**, isolates each application



Mac OS X Grand Central Dispatch (GCD)

- Provides a pool of available threads
- Designers can designate portions of applications, called ***blocks***, that can be dispatched independently and run concurrently
- Concurrency is based on the number of cores available and the thread capacity of the system

Block

- A simple extension to a language
- A block defines a self-contained unit of work
- Enables the programmer to encapsulate complex functions
- Scheduled and dispatched by queues
- Dispatched on a first-in-first-out basis
- Can be associated with an event source, such as a timer, network socket, or file descriptor